一、实验目的与数据说明

1.1 概要

本次实验通过原始数据集的股票收盘价，按照检查平稳性，确定阶数，拟合模型和评估预测的顺序，研究了 ARIMA 模型的建立过程。通过这次实验，基本上可以搞清楚两个问题，一个是历史价格中有多少信息可以用来预测下一天，以及一个在训练数据上表现正常的模型，预测未来时可能会出现什么问题，或者遇上什么棘手的地方。实验中除了建立 ARIMA 模型，还加入了 Naive 和均值两种基线，因此我们可以通过比对不同模型的工作效果来综合性地判断不同模型在预测股票时的效果，从而得到一些见解。

提供的原始数据集的形状是 5306 行和 15 列，包含的数据的时间跨度从 2000 年年初到 2021 年 4 月的月底。Close 列，即股票收盘价字段中，最小值为 67.25，最大值为 1034.00，整个字段没有发现缺失值，也没有日期（索引）重复的情况。按照题目要求，选取 2000 年到 2019 年的 4974 个交易日作为训练集，2020 年全年的 252 个交易日作为测试集。在这个数据集划分下，2021 年的交易记录将不参与模型训练和模型评估。

因此，预测将分为两种方式进行。第一种方式是，先用截止到 2019 年末的数据，一次性预测 2020 年全年的价格。另外一种方式是，每次先预测当天的收盘价，然后等当天的真实收盘价出来后，用这个真实数据去更新模型，再循环往复，继续预测下一个交易日。

1.2 ARIMA 与平稳性

ARIMA(p, d, q) 由自回归，差分和移动平均三部分组成。AR 表示自回归，也就是参考过去几期的数值来估计当前值，其中 p 用于决定往前看多少期。I 表示差分，d 决定做几次差分，例如一次差分就是用当前值减去上一期的值。MA 表示移动平均，利用过去几期的预测误差调整当前预测，其中 q 决定要使用多少期的误差去调整预测。

使用 AR 和 MA 部分时，需要先检查序列是否平稳，也就是平均水平，波动大小和前后数值之间的相关关系是否比较稳定。模型要用过去的数据预测未来，就需要这些关系有一定的延续性。如果股价在不同阶段的平均水平变化很大，从过去估计出的关系就可能不适合后面的数据。因此，建模前要先检查平稳性，必要时可以使用差分，然后在平稳的差分时间序列上进行建模和拟合。

1.2 为什么先检查股价序列

股价经常会连续上涨或下跌，而不同阶段的平均价格也可能相差很大。ARMA 模型需要序列的平均水平和前后数值之间的关系相对稳定，才能用过去估计出的规律预测后面的变化。如果直接用原始股价拟合，模型可能受

到长期涨跌的影响，导致过去拟合得较好，但预测未来时却出现较大偏差。

因此，建模前需要先判断股价序列是否平稳。要检查序列的平稳性，我们通过使用 ADF 检验的方式，通过接受或拒绝原假设 H_0。训练集的股价数据在 ADF 检验后得到的 p 值为 0.11，大于 0.05，未能拒绝其存在单位根的原假设。因此，原始数据集的股价时间序列可能是不平稳的，因此需要考虑使用一阶差分等方式来让序列变得平稳，然后才能进一步预测和分析。

1.3 p 和 q 如何确定

确定差分阶数 d 以后，可以通过 ACF 图和 PACF 图观察差分序列与过去各期的关系。例如，如果 PACF 在 2 阶后截尾，而 ACF 拖尾，就可以初步考虑 AR(2)，即取 $p = 2$ 且 $q = 0$。反过来，如果 ACF 在第 2 阶后截尾，而 PACF 拖尾，就可以初步考虑 MA(2)，即取 $p = 0$ 且 $q = 2$。如果两张图都表现为拖尾，则可以考虑同时包含 AR 和 MA 的模型。不过，实际数据的图形往往没有这么清楚，单靠相关图很难确定唯一的阶数。因此，本文考虑将 p 和 q 分别取 0 至 3 之间的值，再逐一拟合并比较 AIC 和 BIC，然后按 AIC 选择较好的模型。

1.4 残差检查和预测效果

模型拟合后还需要检查剩下的误差是否保留了明显的自相关。如果残差仍能从前几天的误差中找到规律，就意味着模型可能遗漏了一部分信息，即模型在解释上没有用到过去数据中的一些规律。当然，即使这一步没有发现问题，未来价格也可能受到新事件影响，原先估计的关系还可能发生变化。因此，残差诊断完成后，还需要通过在模型未见的测试集上评估模型的预测效果，然后根据情况看是否还得对模型进行进一步的调整。

二、平稳性诊断

首先绘制收盘价随时间变化的曲线，如图 1 所示。在 2000 年到 2019 年的训练集内，股价经历了多次明显的上涨和下跌，有些阶段价格较低，随后又上升到较高水平，之后还会再次回落。2020 年中的交易数据属于训练集，这一年中股价先跌后涨，从 2019 年末的 470 多降至 2020 年 4 月的 250 多，随后逐渐回升，到年末达到 640 多。这样的走势也给预测带来了困难，因为模型既要面对前期的明显下跌，还得准确预测之后的回升。因此，接下来先对训练集进行 ADF 检验评估时间序列的平稳性，再确定差分阶数的取值，为后续建模做准备。

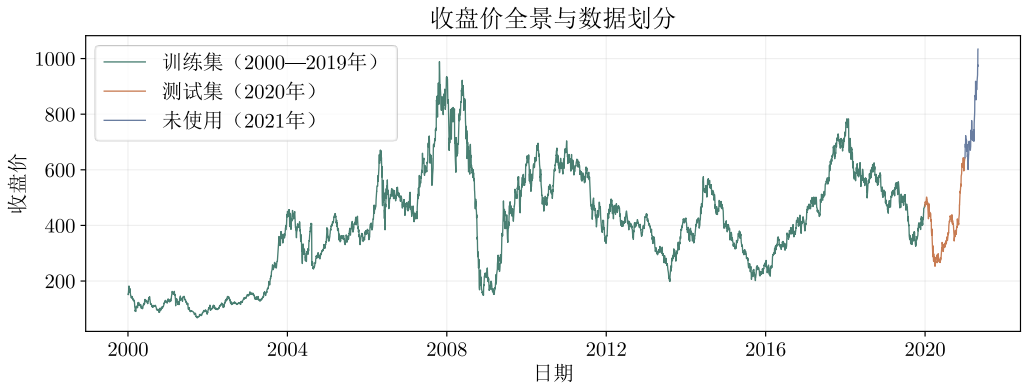


图 1　收盘价全景图

对训练集原始序列，以及它的一阶差分和二阶差分分别做 ADF 检验，对应的结果如下表 1 所示，相应的关键代码如下代码 1 所示。本文采用 0.05 的显著性水平，意味着如果 $p < 0.05$，就认为有足够依据拒绝存在单位根的假设，认为对应的时间序列是平稳的。另外，如果一阶差分后已经通过检验，就优先采用这阶的差分序列，而不继续使用更高阶数的差分序列，哪怕它们的 p 更小。

代码 1　　数据集划分和 ADF 检验

```
train = close.loc[:"2019"]
test = close.loc["2020"]
series = [train, train.diff().dropna(),
train.diff().diff().dropna()]
for values in series:
    result = adfuller(values, regression="c", autolag="AIC")
    print(result[0], result[1])
```

表 1　ADF 检验结果

序列	ADF 统计量	p 值	检验滞后阶数
训练集	−2.53	0.11	9
一阶差分	−21.92	< 0.05	8
二阶差分	−20.13	< 0.05	32

原始训练集对应的时间序列在 ADF 检验后其 p 值为 0.11，大于 0.05，无法拒绝存在单位根的原假设，意味着原始序列不是平稳的，需要差分处理。由于第一阶差分的 p 已经满足 $p < 0.05$ 即它已经是平稳的了，所以我们直接选择差分阶数 $d = 1$。下图 2 则展示了训练集原始序列、一阶差分和二阶差分的时间序列图。其中，一次差分后价格的长期升降变成了围绕零的上下起伏，变化量的标准差为 11.53，而二次差分的标准差反而增至 16.04，曲线起伏更大，所以即便二阶差分也是平稳的，我们仍然取 $d = 1$。

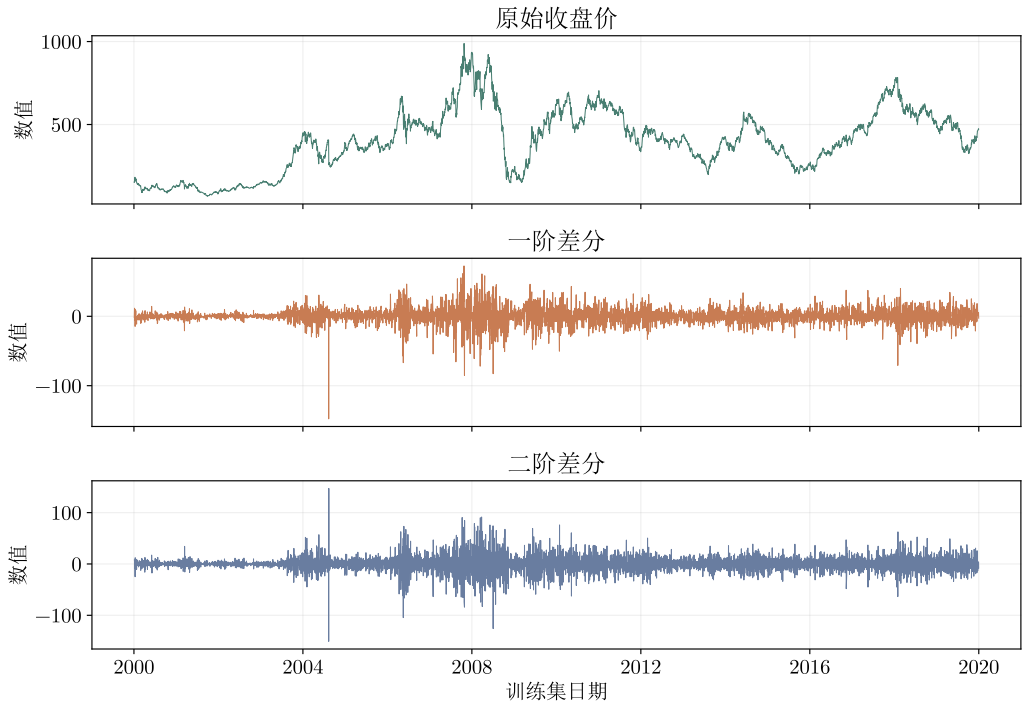


图 2　原始收盘价与差分结果

图 2 中，一阶差分后时间序列中的数值大多围绕零上下波动，原始价格中持续上涨或下跌的走势已经明显减弱。不过，各个时期的起伏大小仍有差别，有些时段变化较小，有些时段则接连出现较大的涨跌。接下来我们将绘制 ACF 和 PACF 图像，可视化地观察这些变化与前几个交易日的关系是什么，再据此确定模型的 p 和 q 应该取多少。

三、自相关结构与定阶

现在绘制前 30 阶的，训练集经过一阶差分的 ACF 图和 PACF 图如下图 3 所示。一阶差分的 ACF(1) 为 0.03，说明相邻两天的交割变化量只有较弱的正相关。下图 3 中，大部分相关系数靠近零，也有少数超出 95%参考范围，但没有出现特别清楚的截尾形态。这次训练数据接近五千条，图中的置信范围比较窄，虽然有些点超出了范围，但相关系数仍然很小。

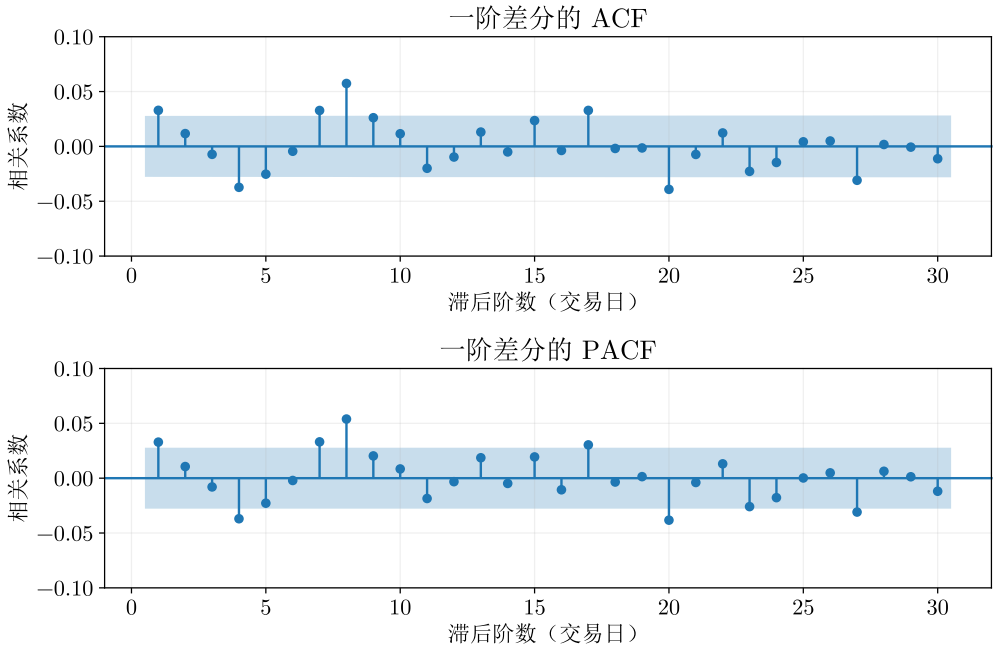


图 3　一阶差分的 ACF 与 PACF

因此，直接从 ACF 图和 PACF 图难以直接确定唯一的 p 和 q 的取值，因为它们并没有表现出明显的截尾或拖尾特征，或者比较模糊，有好几组取值都是可以尝试的。为了获得更好的模型，我们考虑下面的 (p,q) 组合：

表 2　要尝试的 (p,q) 组合

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

然后，我们比较这 16 种组合下，在拟合一阶差分时时的 AIC 和 BIC 值，并从中选出这两项指标最小的，也即 AIC 最小的，作为本文使用的最终模型。如下代码 2 展示了我们通过两层 *for* 循环搜索最佳模型的关键代码，下表 3 则列出了每个参数选取下，拟合出的 ARIMA 模型的 AIC 和 BIC 值。

代码 2　　比较不同阶数的模型

```
# 差分阶数固定为 1, 然后在训练集上比较
# 16 组 (p,q) 取值的 AIC 和 BIC 指标
rows = []
for p in range(4):
    for q in range(4):
        model = ARIMA(train.to_numpy(), order=(p, 1, q),
trend="n").fit()
        rows.append([p, q, model.aic, model.bic])
```

表 3　不同组合下的 AIC 与 BIC 值

(p,q) 组合	阶数	AIC	BIC
(0,0)	(0,1,0)	38434.48	**38440.99**
(0,1)	(0,1,1)	38431.21	38444.23
(0,2)	(0,1,2)	38432.43	38451.96
(0,3)	(0,1,3)	38434.34	38460.39
(1,0)	(1,1,0)	38431.09	38444.11
(1,1)	(1,1,1)	38432.79	38452.33
(1,2)	(1,1,2)	38435.13	38461.18
(1,3)	(1,1,3)	38434.22	38466.78
(2,0)	(2,1,0)	38432.53	38452.06
(2,1)	(2,1,1)	38435.06	38461.11
(2,2)	(2,1,2)	**38413.43**	38445.99
(2,3)	(2,1,3)	38413.84	38452.91
(3,0)	(3,1,0)	38434.21	38460.26
(3,1)	(3,1,1)	38433.53	38466.09
(3,2)	(3,1,2)	38413.88	38452.95
(3,3)	(3,1,3)	38416.10	38461.68

AIC 最低的是 ARIMA(2,1,2)，而 BIC 最低的则是 ARIMA(0,1,0)。我们按事先采用的 AIC 最小的规则，选取 ARIMA(2,1,2) 作为主模型，同时保留较简单的模型 ARIMA(0,1,0) 以用于比较。

与 ARIMA(0,1,0) 相比，ARIMA(2,1,2) 增加了两个 AR 系数和两个 MA 系数，AIC 低了 21.04，BIC 则高了 5.00。两项指标的选择不同，是因为 BIC 对增加参数的惩罚更重，意味着如果按 BIC 为指标，则可能更加倾向于选择更加的简单模型，从而导致未来预测时可能损失一些信息。接下来用测试集的数据比较预测误差，然后观察多出的这 4 个系数可以改善多少的预测。

四、模型拟合与残差诊断

确定使用 ARIMA(2,1,2) 后，用 2000 年到 2019 年的全部训练数据进行拟合。由于前面做了一次差分，模型预测的是每个交易日比前一个交易日涨了多少或跌了多少。再从最后一个已知收盘价开始，再逐日地加上预测的变化量，就能得到未来的价格。本次拟合不加入漂移项，即不在每天的预测变化中额外加上一个固定值。拟合得到的系数，标准误和 p 值见下表 4。

表 4　ARIMA(2,1,2) 系数

参数	coef	std err	$P>\|z\|$
ar.L1	1.25	0.04	<0.001
ar.L2	-0.83	0.04	<0.001
ma.L1	-1.21	0.04	<0.001
ma.L2	0.79	0.05	<0.001
sigma2	132.23	1.08	<0.001

模型预测当天的价格变化时，会先参考前两个交易日的涨跌，分别乘以 1.25 和 -0.83，这就是两个 AR 系数的作用。接着，再根据前两期预测偏高

或偏低的情况进行调整，对应的两个 MA 系数为 -1.21 和 0.79。将这些部分相加，就得到当天价格变化的预测值。这四个系数的 p 值都小于 0.001，在本次拟合中达到了统计显著水平。表中的 sigma2 为 132.23，是模型估计的误差方差，用来描述预测误差的波动大小。

拟合完成后再检查残差，即实际价格减去模型预测价格得到的差值。这里主要是想研究前几天的误差与后面的误差之间是否还有明显联系。由于模型开始计算时需要初始化，本次去掉第一条残差，对剩下的四百多条交易记录进行 LB 检验，分别检查前 10 阶，前 20 阶和前 30 阶的整体相关性。其中，检查前 10 阶就是一起考察相隔 1 至 10 个交易日的残差是否相关，其他的依次类推，对应的结果如下表 5 所示。检验中也对已经估计的 4 个 AR 和 MA 系数作了自由度调整，具体的关键实现见下代码 3 所示。

代码 3　　残差与 LB 检验

```
model = ARIMA(train.to_numpy(), order=(2, 1, 2), trend="n").fit()
# 去掉初始化部分，再检查多个滞后阶数
residuals = model.resid[model.loglikelihood_burn :]
tests = acorr_ljungbox(residuals, lags=[10, 20, 30], model_df=2 + 2)
```

表 5　残差 LB 检验结果

检验到第几阶	LB 统计量	p 值
10	9.04	0.17
20	24.88	0.07
30	36.95	0.08

三个 p 值分别为 0.17、0.07 和 0.08，均大于 0.05，因此在所检查的阶数内，没有发现足够证据认为残差还存在整体自相关。下面的图 4 则是绘制的模型残差及其相关性与分布图，其中残差 ACF 也大多靠近零，与检验结果相符。

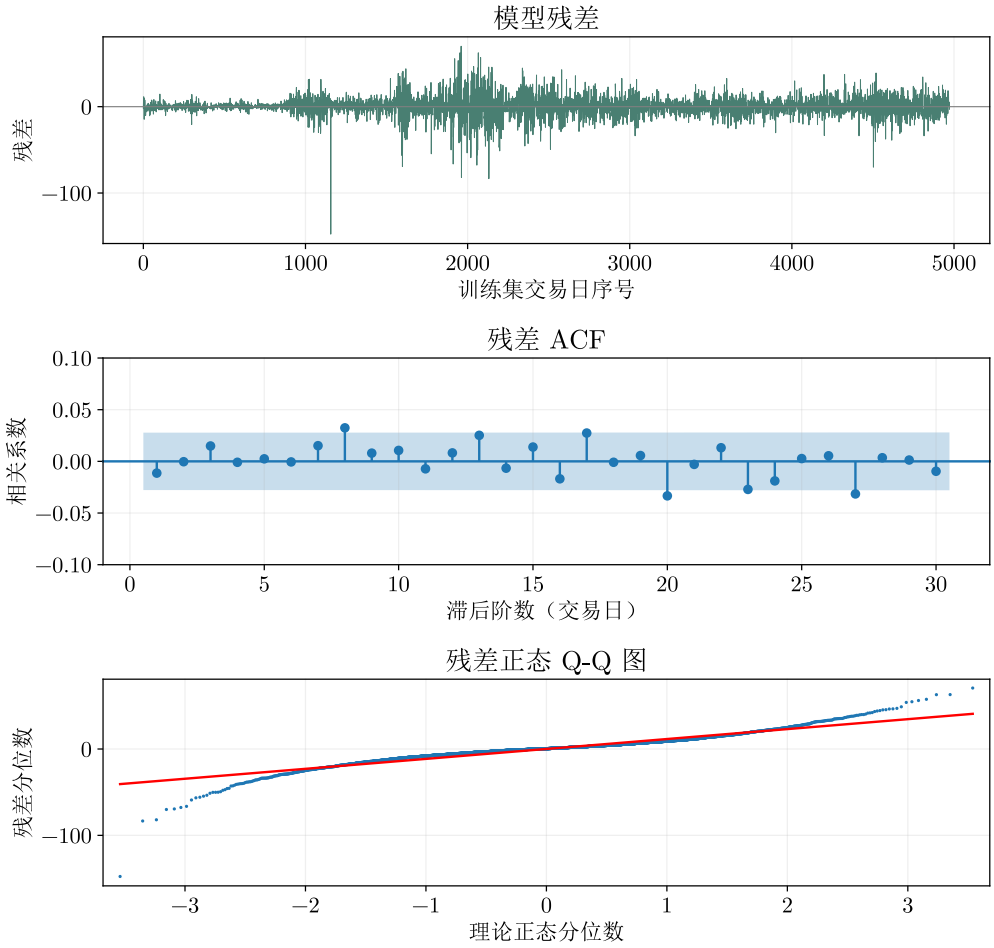


图 4　模型残差及其相关性与分布

从图 4 看，有些时段的残差比较小，有些时段则接连出现较大的残差，说明模型在不同阶段的拟合效果有所差别。Q-Q 图中间的点比较贴近直线，两端却明显偏离，左下方还有一个特别突出的点，这说明模型有时会出现较大的预测偏差，例如其中有一次预测种，预测价格比实际价格高出了很多。结合前面的 LB 检验，可以看出残差之间没有明显的整体自相关，但误差的大小仍会随时间变化，较大的误差也比较突出，这也会影响后面的 95%预测区间，模型可能低估价格大幅上升或下降的可能性，导致实际价格比预想的更容易落在区间之外。

五、预测与误差评估

5.1 全年预测与基线比较

完成模型拟合后，先用 ARIMA(2,1,2) 预测 2020 年的 252 个交易日。这次预测是针对 2020 年全年的，意味着模型通过 2020 年以前的所有训练集进行训练，然后站在 2019 年末一次性算出 2020 年的预测结果，因此预测阶段不会获得逐日更新模型本身。

为了判断 ARIMA 的预测效果，使用 Naïve 和训练均值两种基线模型进行比较。Naïve 直接用前一个交易日的实际收盘价预测当天，因此 2020 年第一个交易日的预测值为 2019 年末的收盘价 472.05，之后每获得一天的真实价格，就用它预测下一个交易日。训练均值基线的做法则是计算 2000 年到 2019 年的平均收盘价，得到约 398.20，并将这个数值作为 2020 年所有交易日的预测值。

三种方法都在同样的测试集上计算 RMSE 和 MAE。RMSE 会让较大的误差占更大比重，MAE 则表示预测价格平均偏离实际价格多少，两项指标都是越小代表模型预测地越好。这三种模型的拟合的关键代码见下代码 4，对应的 RMSE 和 MAE 指标则如下表 6 所示。

代码 4　全年预测与两种基线

```
# 全年 ARIMA 与均值基线不使用 2020 年的实际值
forecast = model.get_forecast(steps=len(test))
arima = forecast.predicted_mean
mean_baseline = train.mean()
# Naive 每天使用前一交易日已经知道的真实收盘价
naive = pd.concat([train.tail(1), test]).shift(1).loc[test.index]
errors = test - arima
rmse = np.sqrt((errors**2).mean())
mae = errors.abs().mean()
```

表 6　测试集上的预测误差比较

预测方法	RMSE	MAE
ARIMA 预测未来一年	123.55	106.50
Naïve 逐日预测	11.28	8.26
训练均值基线	99.54	79.02

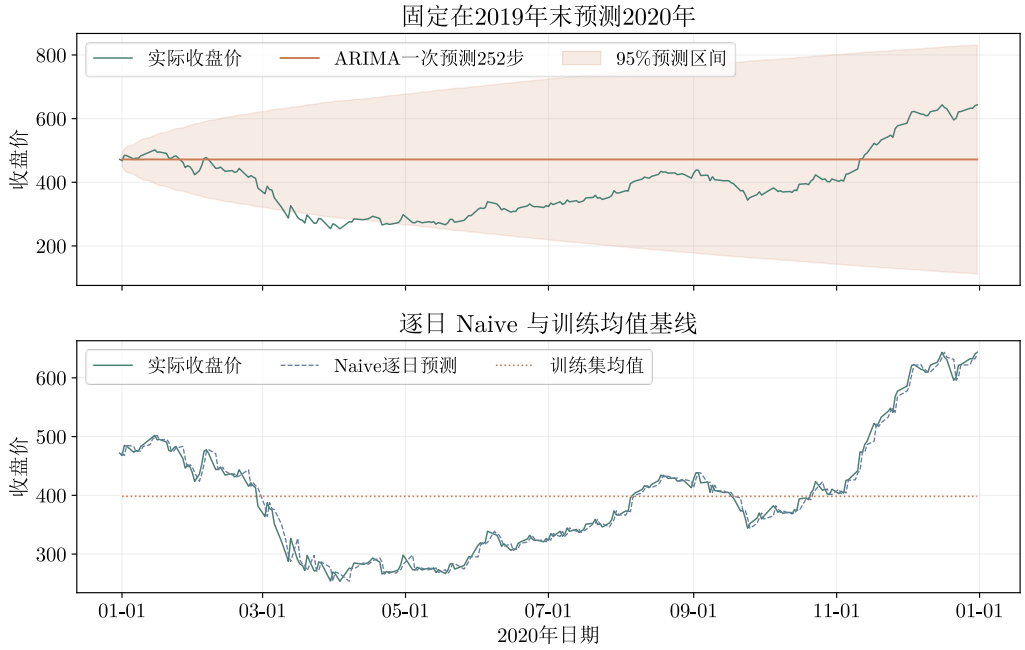


图 5　全年预测与基线比较

图 5 中，ARIMA 的预测曲线几乎是一条水平线，全年都在 472 附近，而实际股价在四月初跌到了 250 左右，之后又逐渐回升，到年末涨至六百多，因此，这表明我们的 ARIMA 模型既没有预测出前面的下跌，也没能预测出后面的上涨。除此外，随着预测时间拉长，95%预测区间也越来越宽，年末已经扩大到 110 多到 830 多的区间，因此模型的预测效果变得很差，很难据此估计价格大致会处于什么水平。

而使用的 Naïve 基线模型，每天都会用最新的真实价格预测下一天，但目前的 ARIMA 模型是一次性预测全年的收盘价，意味着期间没有得到新的价格数据来逐日更新模型，所以这可能成为了预测效果较差的原因。因此，接下来让 ARIMA 也改成每天预测一次，等当天收盘后通过当日的真实价格更新模型，然后继续预测下一个交易日，再与目前的 Naïve 基线模型进行比较，以了解逐日预测的情况下模型的改进效果。

5.2 逐日更新预测

逐日预测时，仍然使用前面训练好的 ARIMA(2,1,2)，但每次预测时候只预测未来一天的收盘价。然后，等真实收盘价出来后，再使用这个实际收盘价更新模型的状态，然后接着预测下一天，如此循环往复。整个过程中，模型系数保持不变，因此模型仍然基于原有的训练集进行训练。这样，我们的 ARIMA 模型和 Naïve 基线算法都能使用截至前一个交易日的真实价格，比较起来也更直接。当前逐日 ARIMA 模型的相关关键代码如下代码 5 所示，该模型和 Naïve 模型在测试集的 RMSE、MAE 指标则如下表 7 所示。

代码 5　　逐日更新并更新状态

```
current = model
rolling = []
for actual in test:
    # 先预测再输入当天真实值
    rolling.append(float(current.forecast(steps=1)[0]))
    # 固定参数（只更新状态）
    current = current.extend([actual])
```

表 7　逐日 ARIMA 与 Naïve 的预测误差

预测方法	RMSE	MAE
ARIMA 逐日更新预测	11.36	8.34
Naïve 逐日预测	11.28	8.26

改成每天预测下一个交易日后，现在的 ARIMA 模型的误差小了很多，RMSE 从 123.55 降到了 11.36，MAE 则从 106.50 降到了 8.34。Naïve 基线算法的 RMSE 和 MAE 分别为 11.28 和 8.26，比目前的 ARIMA 模型稍低一些。从这次的结果看，用前一天的收盘价预测下一天的效果和目前的 ARIMA 模型实际上是差不多的。

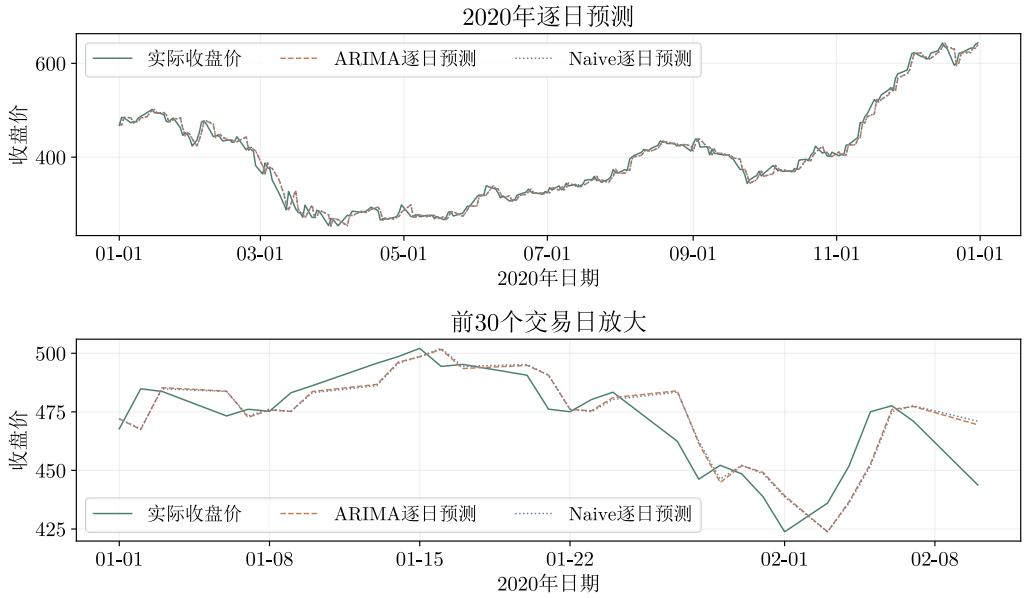


图 6　逐日 ARIMA 与 Naïve 预测比较

图 6 中，现在的逐日 ARIMA 模型和 Naïve 基线模型都比 ARIMA 一次性预测 2020 年整年的数据更加贴近实际的股票收盘价。放大前 30 个交易日后，可以看到，这两个模型对应的预测曲线几乎重合，并且对于价格突然上升或下降的情况，这两个模型都表现出滞后于实际股价变化的情况，必须得等到真实价格更新模型的状态后，下一天的预测才能得到对应的调整。因此，虽然前一天的价格可以为后续的预测提供一定的参考，但对于突然的涨跌，目前的模型仍然有一些滞后性。

5.3 固定起点的 Naïve 预测

接下来让 Naive 也一次预测整个 2020 年，期间不再输入新的真实价格。预测从 2019 年末开始，最后一个已知收盘价是 472.05，所以 2020 年全部两百多个交易日的预测值都是 472.05，画出来就是一条水平线。

这样得到的 RMSE 为 123.71，MAE 为 106.65，与全年一次性预测完的原始 ARIMA 模型的 123.55 和 106.50 差不多，两者的 RMSE 只相差约 0.16。而同一个 ARIMA 改成每天更新后，RMSE 从 123.55 降到了 11.36，改善明显得多。因此，从这次结果上来看，通过及时使用当天的实际收盘价来更新模型的状态，可以很明显的改善模型的预测误差。

六、问题讨论与模型适用范围

6.1 问题一回答

训练集原序列在进行 ADF 检验后，p 值为 0.11，而如果考察包括测试集在内的整个时间序列，则 ADF 检验的 p 值为 0.53。无论是哪种情况，它们的 p 值都大于 0.05，意味着都缺少拒绝单位根假设的依据，因此两种结

果都支持优先考虑差分。首先进行一阶差分，这个差分序列进行 ADF 检验后，p 值已小于 0.05，于是直接取差分阶数 $d = 1$。若直接对原价格拟合平稳 ARMA，参数可能受到长期水平变化影响，得到看似贴近历史的模型，但却难以用于未来的预测。而通过差分，分析对象变成了相邻交易日的价格变化，减少了长期水平变化带来的影响。

6.2 问题二回答

一阶差分的 ACF(1) 只有 0.03，说明相邻两天的价格变化几乎没有线性关系，暗示每天的涨跌接近白噪声过程。也就是说，昨天的股市收盘价的改变量对判断今天的改变量在预测上帮助不大。因此单靠历史价格，模型能找到的短期规律比较少，很难准确预测明天的股市收盘价是涨还是跌，并且改变多少也不太清楚。这也解释了为什么 ARIMA 的逐日预测模型的预测效果与直接沿用前一天价格的 Naïve 基线模型基本相同。

6.3 问题三回答

如果考虑选择 AIC 指标最小的模型，则选择的是 ARIMA(2,1,2) 这个模型，而如果是考虑 BIC 最小的模型，则对应的是 ARIMA(0,1,0) 这个模型，这两种选法的取舍各不相同。

ARIMA(2,1,2) 多了四个系数，AIC 只降低了 21.04，再看 2020 年的预测结果，它的 RMSE 为 123.55，而 ARIMA(0,1,0) 为 123.71，只相差约 0.16。这里的 ARIMA(0,1,0) 做法很简单，就是把 2019 年末的收盘价 472.05 作为全年每一天的预测值。两种模型的误差差不多，说明增加四个系数后，在全年预测上的效果改善很小。

因此，这也意味着更加复杂的模型可能并不一定是效果非常显著的模型，在部分场景下，更加复杂的模型很可能只能提供很小的一部分效果增益，因此实际应该使用什么模型，还取决于这些模型在预测未见的测试集上的效果改善如何，而不能盲目的认为更加复杂的模型就一定可以大幅度地改善模型的预测效果。

6.4 问题四回答

这两件事不矛盾。残差检验通过说明在所检查的阶数内，剩下的误差没有明显的整体自相关，很难再根据前几天的误差找到修正预测的规律。本次 LB 检验的三个 p 值都大于 0.05，意味着模型通过了这项检查。

然而，预测未来还需要面对新的变化。即使模型已经用好了历史数据中的规律，也很难提前知道突发事件会让价格怎样变化。如果每天收盘价的变化量本身就有很强的随机性，模型也可能留下接近白噪声的残差，却仍然无法很准确地预知第二天的股价。

因此，残差检查通过，能支持模型没有遗漏明显自相关这一判断，但并

不能保证未来的预测误差一定很小。例如，5.1 中的全年预测的模型的 RMSE 就达到 123.55，就属于这种情况。

6.5 问题五回答

Naïve 基线算法的 RMSE 能达到 11.28 这样的低值，是因为它每天都会跟着新的真实价格调整，而我们的原始 ARIMA 模型是一次性预测整个 2020 年的股市收盘价，意味着它只能利用 2019 年末的信息。

随着预测越来越远，原始 ARIMA 模型对未来价格变化的估计逐渐接近零，又没有漂移项，累加后的价格便停在约 471.78 附近。将每天的收盘价用于更新 ARIMA 的模型状态后，RMSE 降至 11.36，与 Naïve 基线模型的预测误差基本差不多了，意味着预测跨度和更新方式是本次误差差距的重要原因。

当然，Naïve 模型能取得较低的一步误差，也与相邻两天的股市收盘价通常不会完全脱离前一天的价格有关。

6.6 问题六回答

ARIMA 比较适合经过差分后能够趋于平稳，前后数据之间存在一定相关性，并且这种关系能保持一段时间的序列。例如，过去几天的变化对下一天仍有参考价值，就可以尝试用 ARIMA 做短期预测。本次一次预测全年的 RMSE 为 123.55，改成逐日更新后降到了 11.36，说明对这组股价数据来说，及时用上新价格比长期沿用不变的训练数据具有较大的改善效果。

2020 年的疫情改变了市场环境，过去价格中的规律很难反映这种突然出现的变化。模型在 2019 年末只看过历史收盘价，无法提前知道疫情的发展及其对市场的影响，所以图 5 中的下跌和后续回升都没有预测出来。

因此，遇到政策调整或突发事件，如果模型只结合过去的价格进行预测就没办法应对未来的股价走势的急剧变化，因此还得结合当时的市场情况动态调整。例如，可以在 ARIMAX 模型中加入预测时已经公布的经济或行业数据，让模型多掌握一些判断依据。如果想了解的是价格波动为什么有些时候比较大而有些时候比较小，则可以考虑用 GARCH 模型分析。在这种存在外部因素剧烈影响的情况下，单纯增加 ARIMA 的阶数，模型用到的仍然是过去的价格预测未来的序列，缺少的外部信息还需要另外补充。

七、实验总结与体会

通过这次实验，我们熟悉了 ARIMA 建模的基本流程，包括检查平稳性，确定差分次数，选择阶数，以及拟合后的残差检查和预测评估。其中，价格和价格变化量的区分值得注意，虽然相邻两天的股价往往比较接近，但一次差分后的 ACF(1) 只有 0.03，说明昨天的股价收盘价的变化量对判断今天的变化帮助并不大。实际操作中，通过对原始时间序列进行 ADF 检验，最终

确定采用一阶差分，然后再结合 ACF 图和 PACF 图，以及 AIC 指标来选择模型，最后检查残差。

原始的 ARIMA 模型和每天收盘价出来后逐日更新模型状态的 ARIMA 模型的结果差别也很明显，从本次实验可以看到，同一个 ARIMA 模型，一次性预测未来全年时的RMSE为123.55，但每天输入新的真实价格后，RMSE降到了 11.36。另外，ARIMA(2,1,2) 比简单模型多了四个系数，AIC 降低了 21.04，但全年预测的 RMSE 只改善约 0.16。在改为根据每天的收盘价更新模型状态后，它与 Naïve 基线模型的误差也很接近。因此，这次实验保留 Naïve 基线模型作为参照很有帮助，可以很容易地评估复杂模型相对于简单的模型的改善程度。

残差检查与实际预测之间出现的问题也是值得关注的。本次实验进行LB检验的三个 p 值都大于 0.05，说明残差中已经没有明显的整体自相关，但预测 2020 年时，模型给出的价格始终停留在 472 附近，与实际的下降和上升的相差很大。这个现象意味着检验和预测是两个完全不同的概念。其中，前者可以帮助检查模型是否遗漏了历史数据中的相关关系，而后者要看这些关系用到未来时，究竟能预测得多准。通过这次实验，以及对我们的 ARIMA 模型与基线模型的比较，让我了解到建模时选择基线模型比较当前模型的改进效果也是非常重要的，这样至少可以提供一个基本的参照。

八、AI 工具使用声明

本人未与他人讨论本实验，但使用了 OpenAI Codex 的 GPT 6.1 Sol 模型辅助代码编写与修改，也包括图表绘制、结果分析和报告文字的草拟。在这基础上，再经过大量人工修改、复核和排版后撰写出本文。

九、附录

`analysis.py` (程序入口点)

```python
import numpy as np
from common import Config
from step_one_stationarity import run_step_one
from step_two_order import run_step_two
from step_three_diagnosis import run_step_three
from step_four_forecast import run_step_four

if __name__ == "__main__":
    np.random.seed(Config.SEED)
    train, test, adf = run_step_one()
    order, grid = run_step_two(train)
    model, coefficients, tests = run_step_three(train, order)
    predictions, metrics = run_step_four(train, test, model)
```

`common.py` (读取数据与设置)

```python
from pathlib import Path
import pandas as pd
```

```python
class Config:
    ROOT: Path = Path(__file__).resolve().parent.parent
    FIGURES: Path = ROOT / "figures"
    SEED: int = 42

def load_close() -> pd.Series:
    """按交易日顺序读取收盘价"""
    data = pd.read_csv(Config.ROOT / "TATASTEEL.csv",
parse_dates=["Date"])
    return data.sort_values("Date").set_index("Date")["Close"]
```

plot.py (绘制实验图表)

```python
from typing import Any

import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import pandas as pd
from statsmodels.graphics.gofplots import qqplot
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

from common import Config

class PlotConfig:
    STYLE: dict[str, Any] = {
        "font.family": ["Latin Modern Math", "SimSun"],
        "mathtext.fontset": "cm",
        "axes.formatter.use_mathtext": True,
        "axes.unicode_minus": False,
        "svg.fonttype": "path",
        "font.size": 14,
    }
    COLORS: list[str] = ["#497f72", "#c87c53", "#697da0"]

plt.rcParams.update(PlotConfig.STYLE)

def save_figure(name: str) -> None:
    Config.FIGURES.mkdir(exist_ok=True)
    plt.tight_layout()
    plt.savefig(Config.FIGURES / name, format="svg",
bbox_inches="tight")
    plt.close()

def plot_overview(close: pd.Series) -> None:
    """查看训练、测试及未使用部分的收盘价"""
    fig, ax = plt.subplots(figsize=(10, 4))
    for series, name, color in zip(
        [close.loc[:"2019"], close.loc["2020"], close.loc["2021":]],
        ["训练集（2000–2019 年）", "测试集（2020 年）", "未使用（2021
年）"],
        PlotConfig.COLORS,
    ):
        ax.plot(series.index, series, color=color, lw=0.9,
label=name)
    ax.set(title="收盘价全景与数据划分", xlabel="日期", ylabel="收盘价
")
```

```
    ax.legend(loc="upper left")
    ax.grid(alpha=0.2)
    save_figure("01 收盘价全景.svg")

def plot_differences(train: pd.Series) -> None:
    """比较原始价格与两种差分结果"""
    fig, axes = plt.subplots(3, 1, figsize=(10, 7), sharex=True)
    series_list = [train, train.diff().dropna(),
train.diff().diff().dropna()]
    for ax, series, title, color in zip(
        axes, series_list, ["原始收盘价", "一阶差分", "二阶差分"],
PlotConfig.COLORS
    ):
        ax.plot(series.index, series, color=color, lw=0.7)
        ax.set_title(title)
        ax.set_ylabel("数值")
        ax.grid(alpha=0.2)
    axes[-1].set_xlabel("训练集日期")
    save_figure("02 差分与平稳性.svg")

def plot_correlations(difference: pd.Series) -> None:
    """只用训练集的一阶差分查看前 30 阶相关性"""
    fig, axes = plt.subplots(2, 1, figsize=(9, 6))
    plot_acf(difference, lags=30, zero=False, ax=axes[0], title="一
阶差分的 ACF")
    plot_pacf(
        difference,
        lags=30,
        zero=False,
        method="ywm",
        ax=axes[1],
        title="一阶差分的 PACF",
    )
    for ax in axes:
        ax.set(xlabel="滞后阶数（交易日）", ylabel="相关系数")
        ax.set_ylim(-0.10, 0.10)
        ax.grid(alpha=0.2)
    save_figure("03 一阶差分相关图.svg")

def plot_residuals(residuals: pd.Series) -> None:
    """检查残差的变化，自相关和正态分布偏离情况"""
    fig, axes = plt.subplots(3, 1, figsize=(9, 8.6))
    axes[0].plot(residuals.to_numpy(), color=PlotConfig.COLORS[0],
lw=0.6)
    axes[0].axhline(0, color="gray", lw=0.7)
    axes[0].set(title="模型残差", xlabel="训练集交易日序号", ylabel="
残差")
    plot_acf(residuals, lags=30, zero=False, ax=axes[1], title="残差
ACF")
    axes[1].set(xlabel="滞后阶数（交易日）", ylabel="相关系数")
    axes[1].set_ylim(-0.10, 0.10)
    qqplot(residuals, line="s", ax=axes[2], marker=".",
markersize=2)
```

```
    axes[2].set(title="残差正态 Q-Q 图", xlabel="理论正态分位数",
ylabel="残差分位数")
    for ax in axes:
        ax.grid(alpha=0.2)
    save_figure("04 模型残差诊断.svg")

def set_test_axis(ax: plt.Axes) -> None:
    """统一 2020 年预测图的日期刻度"""
    ax.xaxis.set_major_locator(mdates.MonthLocator(interval=2))
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%m-%d"))
    ax.set(xlabel="2020 年日期", ylabel="收盘价")
    ax.grid(alpha=0.2)

def plot_forecast(
    train: pd.Series,
    test: pd.Series,
    predictions: pd.DataFrame,
    interval: pd.DataFrame,
) -> None:
    """比较一次全年预测与基线模型"""
    fig, axes = plt.subplots(2, 1, figsize=(10, 6.5), sharex=True)
    actual = pd.concat([train.tail(1), test])
    for ax in axes:
        ax.plot(
            actual.index, actual, color=PlotConfig.COLORS[0], lw=1,
label="实际收盘价"
        )
        set_test_axis(ax)
    axes[0].plot(
        test.index,
        predictions["ARIMA 一次预测"],
        color=PlotConfig.COLORS[1],
        lw=1.3,
        label="ARIMA 一次预测 252 步",
    )
    axes[0].fill_between(
        test.index,
        interval.iloc[:, 0],
        interval.iloc[:, 1],
        color=PlotConfig.COLORS[1],
        alpha=0.15,
        label="95%预测区间",
    )
    axes[0].set_title("固定在 2019 年末预测 2020 年")
    axes[0].set_xlabel("")
    axes[1].plot(
        test.index,
        predictions["Naive 逐日预测"],
        color=PlotConfig.COLORS[2],
        lw=0.9,
        ls="--",
        label="Naive 逐日预测",
    )
    axes[1].plot(
        test.index,
```

```
        predictions["训练均值"],
        color=PlotConfig.COLORS[1],
        lw=1.1,
        ls=":",
        label="训练集均值",
    )
    axes[1].set_title("逐日 Naive 与训练均值基线")
    for ax in axes:
        ax.legend(loc="upper left", ncol=3, fontsize=13)
    save_figure("05 全年预测比较.svg")

def plot_rolling(test: pd.Series, rolling: pd.Series, naive:
pd.Series) -> None:
    """在相同的逐日预测条件下比较两种方法"""
    fig, axes = plt.subplots(2, 1, figsize=(10, 6))
    # 全年图观察整体变化，前 30 个交易日放大查看相近的预测
    for ax, count, title in zip(
        axes, [len(test), 30], ["2020 年逐日预测", "前 30 个交易日放大"]
    ):
        for series, name, color, style in zip(
            [test, rolling, naive],
            ["实际收盘价", "ARIMA 逐日预测", "Naive 逐日预测"],
            PlotConfig.COLORS,
            ["-", "--", ":"],
        ):
            ax.plot(
                test.index[:count],
                series.iloc[:count],
                color=color,
                ls=style,
                lw=1,
                label=name,
            )
        ax.set(title=title, xlabel="2020 年日期", ylabel="收盘价")
        ax.xaxis.set_major_formatter(mdates.DateFormatter("%m-%d"))
        ax.grid(alpha=0.2)
        ax.legend(ncol=3, fontsize=13)
    save_figure("06 逐日预测比较.svg")
```

step_one_stationarity.py (检查平稳性)

```
import pandas as pd
from statsmodels.tsa.stattools import adfuller
from common import load_close
from plot import plot_overview, plot_differences

def run_step_one() -> tuple[pd.Series, pd.Series, pd.DataFrame]:
    """划分训练与测试数据"""
    close = load_close()
    train, test = close.loc[:"2019"], close.loc["2020"]
    print(f"数据共{len(close)}条，训练集{len(train)}条，测试集
{len(test)}条。")
    print(
        f"收盘价缺失{close.isna().sum()}条，重复日期
{close.index.duplicated().sum()}条。"
```

```
    )
    rows = []
    series = [train, train.diff().dropna(),
train.diff().diff().dropna()]
    for name, values in zip(["原序列", "一次差分", "二次差分"],
series):
        statistic, p_value, lag, *_ = adfuller(values,
regression="c", autolag="AIC")
        rows.append([name, statistic, p_value, lag])
    adf = pd.DataFrame(rows, columns=["序列", "ADF 统计量", "p 值", "
滞后阶数"])
    print("\n 一、训练集平稳性检验\n" +
adf.round(6).to_string(index=False))
    plot_overview(close)
    plot_differences(train)
    return train, test, adf
```

step_two_order.py (搜索模型阶数)

```
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import acf
from plot import plot_correlations

def run_step_two(train: pd.Series) -> tuple[tuple[int, int, int],
pd.DataFrame]:
    """确定差分阶数后，比较十六组 (p,q) 取值的 AIC 和 BIC 指标"""
    difference = train.diff().dropna()
    print(f"\n 二、定阶：一次差分的 ACF(1)={acf(difference,
nlags=1)[1]:.6f}")
    rows = []
    for p in range(4):
        for q in range(4):
            model = ARIMA(train.to_numpy(), order=(p, 1, q),
trend="n").fit()
            rows.append([p, q, model.aic, model.bic,
model.mle_retvals["converged"]])
    grid = pd.DataFrame(rows, columns=["p", "q", "AIC", "BIC", "收敛
"])
    valid = grid.loc[grid["收敛"]]
    best = valid.loc[valid["AIC"].idxmin()]
    order = (int(best["p"]), 1, int(best["q"]))
    print(grid.round(3).to_string(index=False))
    print(f"按 AIC 选择 ARIMA{order}，模型不包含漂移项。")
    plot_correlations(difference)
    return order, grid
```

step_three_diagnosis.py (拟合与检查残差)

```
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA, ARIMAResultsWrapper
from statsmodels.stats.diagnostic import acorr_ljungbox
from plot import plot_residuals

def run_step_three(
    train: pd.Series, order: tuple[int, int, int]
```

```
) -> tuple[ARIMAResultsWrapper, pd.DataFrame, pd.DataFrame]:
    """拟合模型"""
    model = ARIMA(train.to_numpy(), order=order, trend="n").fit()
    coefficients = pd.DataFrame(
        {"coef": model.params, "std err": model.bse, "P>|z|":
model.pvalues},
        index=model.param_names,
    )
    burn = model.loglikelihood_burn
    residuals = pd.Series(model.resid[burn:],
index=train.index[burn:])
    p, _, q = order
    tests = acorr_ljungbox(residuals, lags=[10, 20, 30], model_df=p
+ q)
    print("\n 三、模型系数\n" +
coefficients.to_string(float_format="%.6g"))
    print(
        "残差 Ljung-Box 检验（已扣除 AR、MA 参数的自由度）：\n" +
tests.round(6).to_string()
    )
    plot_residuals(residuals)
    return model, coefficients, tests
```

step_four_forecast.py（预测并比较误差）

```
import numpy as np
import pandas as pd
from statsmodels.tsa.arima.model import ARIMAResultsWrapper
from plot import plot_forecast, plot_rolling

def run_step_four(
    train: pd.Series, test: pd.Series, model: ARIMAResultsWrapper
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """比较全年预测和逐日预测，逐日更新状态但不重新估计参数"""
    forecast = model.get_forecast(steps=len(test))
    predictions = pd.DataFrame(index=test.index)
    predictions["ARIMA 一次预测"] = forecast.predicted_mean
    predictions["Naive 逐日预测"] = (
        pd.concat([train.tail(1), test]).shift(1).loc[test.index]
    )
    predictions["训练均值"] = train.mean()
    interval = pd.DataFrame(forecast.conf_int(), index=test.index)
    plot_forecast(train, test, predictions, interval)

    # 每次先预测，再接收当天真实值
    rolling = []
    current = model
    for actual in test:
        rolling.append(float(current.forecast(steps=1)[0]))
        current = current.extend([actual])
    predictions["ARIMA 逐日预测"] = rolling
    predictions["Naive 固定起点"] = train.iloc[-1]
    rows = []
    for name in predictions:
        errors = test - predictions[name]
```

```
        rows.append([name, np.sqrt((errors**2).mean()),
errors.abs().mean()])
    metrics = pd.DataFrame(rows, columns=["方法", "RMSE", "MAE"])
    print("\n 四、2020 年预测误差\n" +
metrics.round(4).to_string(index=False))
    plot_rolling(test, predictions["ARIMA 逐日预测"],
predictions["Naive 逐日预测"])
    return predictions, metrics
```